SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A Lokanath Reddy	**Reg. No.:**	192425321
Section / Batch:	2024	**Date:**	

Code of Conduct

I A Lokanath Reddy/192425321 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %

Faculty In-charge	Course Coordinator	Program Director
Dr Kumaraguruban T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Q1 Solution HMM POS Tagging

1. Separate Each Sentence into Words and POS Tags

Sentence	Words	POS Tags
The boy eats rice	The, boy, eats, rice	DT, NN, VBZ, NN
The girl drinks milk	The, girl, drinks, milk	DT, NN, VBZ, NN
A cat drinks milk	A, cat, drinks, milk	DT, NN, VBZ, NN
The dog chases cat	The, dog, chases, cat	DT, NN, VBZ, NN
A teacher teaches students	A, teacher, teaches, students	DT, NN, VBZ, NNS
Students study English	Students, study, English	NNS, VBP, NN
Birds fly high	Birds, fly, high	NNS, VBP, RB
Children play games	Children, play, games	NNS, VBP, NNS

2. Compute the Emission Probability

Formula

$P(\text{word} \mid \text{tag}) = \frac{\text{Count(word,tag)}}{\text{Count(tag)}}$

POS Tag Counts

POS Tag	Count
DT	5
NN	10
VBZ	5
NNS	5
VBP	3
RB	1

Emission Probabilities

Word	Tag	Calculation	Probability
The	DT	3/5	0.60
A	DT	2/5	0.40
boy	NN	1/10	0.10
girl	NN	1/10	0.10
rice	NN	1/10	0.10
milk	NN	2/10	0.20
cat	NN	2/10	0.20
dog	NN	1/10	0.10
teacher	NN	1/10	0.10
English	NN	1/10	0.10
eats	VBZ	1/5	0.20
drinks	VBZ	2/5	0.40
chases	VBZ	1/5	0.20
teaches	VBZ	1/5	0.20
students	NNS	1/5	0.20
Students	NNS	1/5	0.20
Birds	NNS	1/5	0.20
Children	NNS	1/5	0.20
games	NNS	1/5	0.20
study	VBP	1/3	0.33
fly	VBP	1/3	0.33
play	VBP	1/3	0.33
high	RB	1/1	1.00

Examples

$P(\text{The} \mid \text{DT}) = \frac{3}{5} = 0.60$ $P(\text{cat} \mid \text{NN}) = \frac{2}{10} = 0.20$ $P(\text{drinks} \mid \text{VBZ}) = \frac{2}{5} = 0.40$

Therefore, the emission probabilities are obtained from the frequency of each word under its corresponding POS tag.

3. Compute the Transition Probability

Formula

$P(Tag_j \mid Tag_i) = \frac{\text{Count}(Tag_i \rightarrow Tag_j)}{\text{Total outgoing transitions from } Tag_i}$

Transition Counts

DT → NN = 5
NN → VBZ = 5
VBZ → NN = 4
VBZ → NNS = 1
NNS → VBP = 3
VBP → NN = 1
VBP → RB = 1
VBP → NNS = 1

Transition Probabilities

From	To	Calculation	Probability
DT	NN	5/5	**1.00**
NN	VBZ	5/5	**1.00**
VBZ	NN	4/5	**0.80**
VBZ	NNS	1/5	**0.20**
NNS	VBP	3/3	**1.00**
VBP	NN	1/3	**0.33**
VBP	RB	1/3	**0.33**
VBP	NNS	1/3	**0.33**

Important

For example:

$P(VBZ \mid NN) = \frac{5}{5} = 1.00$

because there are **5 outgoing NN transitions**, and all 5 go to VBZ.

Similarly:

$P(VBP \mid NNS) = \frac{3}{3} = 1.00$

4. Recreate the Hidden Markov Model Using Python Dictionaries

The following program automatically obtains the HMM probabilities from the given training corpus.

```
# Training corpus

training_data = [
    (["The", "boy", "eats", "rice"],
     ["DT", "NN", "VBZ", "NN"]),

    (["The", "girl", "drinks", "milk"],
     ["DT", "NN", "VBZ", "NN"]),

    (["A", "cat", "drinks", "milk"],
     ["DT", "NN", "VBZ", "NN"]),

    (["The", "dog", "chases", "cat"],
     ["DT", "NN", "VBZ", "NN"]),
```

```
    (["A", "teacher", "teaches", "students"],
     ["DT", "NN", "VBZ", "NNS"]),

    (["Students", "study", "English"],
     ["NNS", "VBP", "NN"]),

    (["Birds", "fly", "high"],
     ["NNS", "VBP", "RB"]),

    (["Children", "play", "games"],
     ["NNS", "VBP", "NNS"])
]


tag_count = {}
emission_count = {}
transition_count = {}


# Count tags and word-tag pairs
for words, tags in training_data:

    for word, tag in zip(words, tags):

        tag_count[tag] = tag_count.get(tag, 0) + 1

        if tag not in emission_count:
            emission_count[tag] = {}

        emission_count[tag][word] = (
            emission_count[tag].get(word, 0) + 1
        )


# Count transitions
for words, tags in training_data:

    for i in range(len(tags) - 1):

        current = tags[i]
        next_tag = tags[i + 1]

        if current not in transition_count:
            transition_count[current] = {}

        transition_count[current][next_tag] = (
            transition_count[current].get(next_tag, 0) + 1
        )


# Emission probabilities
emission = {}

for tag in emission_count:

    emission[tag] = {}

    for word in emission_count[tag]:

        emission[tag][word] = (
            emission_count[tag][word] / tag_count[tag]
        )
```

```
# Transition probabilities
transition = {}

for tag in transition_count:

    transition[tag] = {}

    total = sum(transition_count[tag].values())

    for next_tag in transition_count[tag]:

        transition[tag][next_tag] = (
            transition_count[tag][next_tag] / total
        )


print("Emission Probabilities:")
print(emission)

print("\nTransition Probabilities:")
print(transition)
```

5. Implement the Viterbi Algorithm

The Viterbi algorithm finds the **most probable sequence of hidden POS tags** by considering both:

Previous probability×Transition probability×Emission probability
Python Program

```
# Viterbi Algorithm

sentence = ["The", "cat", "drinks", "milk"]

tags = list(emission.keys())

viterbi = []
backpointer = []


# First word
first_row = {}
first_back = {}

for tag in tags:

    if sentence[0] in emission[tag]:
        first_row[tag] = emission[tag][sentence[0]]
    else:
        first_row[tag] = 0

    first_back[tag] = None

viterbi.append(first_row)
backpointer.append(first_back)


# Remaining words
for i in range(1, len(sentence)):

    current_row = {}
    current_back = {}

    word = sentence[i]
```

```
    for current_tag in tags:

        best_probability = 0
        best_previous_tag = None

        for previous_tag in tags:

            if previous_tag in transition:

                if current_tag in transition[previous_tag]:

                    transition_probability = (
                        transition[previous_tag][current_tag]
                    )

                    if word in emission[current_tag]:

                        emission_probability = (
                            emission[current_tag][word]
                        )

                        probability = (
                            viterbi[i - 1][previous_tag]
                            * transition_probability
                            * emission_probability
                        )

                        if probability > best_probability:

                            best_probability = probability
                            best_previous_tag = previous_tag

        current_row[current_tag] = best_probability
        current_back[current_tag] = best_previous_tag

    viterbi.append(current_row)
    backpointer.append(current_back)


# Find best final tag
last_tag = max(
    viterbi[-1],
    key=viterbi[-1].get
)

predicted_tags = [last_tag]


# Backtracking
for i in range(len(sentence) - 1, 0, -1):

    last_tag = backpointer[i][last_tag]

    predicted_tags.append(last_tag)


predicted_tags.reverse()


print("\nSentence :", " ".join(sentence))
print("POS Tags :", " ".join(predicted_tags))
```

Viterbi Calculation

For the input:

The cat drinks milk
Step 1 — The

Only DT has an emission probability for The.

$P(\text{The} \mid \text{DT}) = 0.60$

So:

The → DT

Probability:

0.60

Step 2 — cat

Transition:

$P(\text{NN} \mid \text{DT}) = 1.00$

Emission:

$P(\text{cat} \mid \text{NN}) = 0.20$

Therefore:

$0.60 \times 1.00 \times 0.20 = 0.12$

So:

The → DT
cat → NN

Step 3 — drinks

Transition:

$P(\text{VBZ} \mid \text{NN}) = 1.00$

Emission:

$P(\text{drinks} \mid \text{VBZ}) = 0.40$

Therefore:

$0.12 \times 1.00 \times 0.40 = 0.048$

So:

The → DT
cat → NN

drinks → VBZ

Step 4 — milk

Transition:

P(NN | VBZ)=0.80

Emission:

P(milk | NN)=0.20

Therefore:

0.048×0.80×0.20=0.00768

Final path:

DT → NN → VBZ → NN

6. Predict the POS Tag Sequence

Word	Predicted POS Tag
The	DT
cat	NN
drinks	VBZ
milk	NN

Final Output
Sentence : The cat drinks milk
POS Tags : DT NN VBZ NN

Result

The Hidden Markov Model was successfully recreated from the manually POS-tagged training corpus. The emission and transition probabilities were calculated, and the Viterbi algorithm was implemented without using any built-in NLP or HMM library. The predicted POS tag sequence for **"The cat drinks milk"** is:

DT NN VBZ NN